1 Genetic Algorithms and Evolutionary Computation

Genetic Algorithms (GAs) provide a robust learning method motivated by the process of natural selection. Unlike gradient-based methods, GAs conduct a randomized, parallel, hill-climbing search through a hypothesis space without requiring the error function to be differentiable.

Evolutionary Operators

Before examining the formal algorithm, we define the mechanisms that allow a population of bit strings (chromosomes) to evolve.

2 Selection: Survival of the Fittest

Selection is the process of choosing individuals from the current population P to be placed into the mating pool P_s. The goal is to bias the search toward more "fit" hypotheses while maintaining enough genetic diversity to explore the search space effectively.

2.1 Fitness Proportionate Selection (Roulette Wheel)

In this method, the probability that a hypothesis h_i will be selected is directly proportional to its fitness score relative to the rest of the population.

$$\Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^{p} Fitness(h_j)}$$

Example: Suppose we have a small population of two individuals:

- h_1 with $Fitness(h_1) = 10$
- h_2 with $Fitness(h_2) = 2$

The total fitness is $10+2 = 12$. The probability of picking h_1 is $10/12 \approx 83.3\%$, while the probability for h_2 is $2/12 \approx 16.7\%$. Consequently, h_1 is **five times** more likely to be selected than h_2.

2.2 Tournament Selection

Tournament selection is a more competitive approach that often helps prevent a single highly fit individual from dominating the population too quickly (a problem known as *crowding*).

The Procedure:

1. Randomly pick two individuals (h_i and h_j) from the population with uniform probability.
2. Generate a random number $r \in [0, 1]$.
3. If $r < p$ (where p is a parameter, e.g., 0.8), select the individual with the **higher** fitness.
4. Otherwise, select the individual with the **lower** fitness.

By occasionally selecting the weaker individual, this method **maintains genetic diversity**, allowing potentially useful genetic material in less fit individuals to persist in the population for further exploration.

2.3 Rank Selection

Rank Selection is a selection mechanism designed to mitigate the **crowding** problem associated with Fitness Proportionate Selection. Crowding occurs when a "super-individual" with a fitness score significantly higher than the rest of the population is chosen so frequently that it dominates the mating pool, leading to a rapid loss of genetic diversity and premature convergence.

2.4 The Ranking Mechanism

In Rank Selection, the absolute values of the fitness scores are discarded. Instead, the population is sorted according to fitness, and each individual is assigned a rank $R \in \{1, \ldots, p\}$, where p is the population size.

- **Sorting:** The least fit individual is assigned Rank 1, and the most fit individual is assigned Rank p.
- **Handling Ties:** If two hypotheses h_i and h_j have identical fitness values, they are typically assigned the **same rank**. This ensures that individuals with equal performance have an equal probability of being selected.

2.5 Selection Probability

Once ranks are assigned, the probability of selecting an individual h_i is determined by its rank $R(h_i)$ rather than its raw fitness:

$$\Pr(h_i) = \frac{R(h_i)}{\sum_{j=1}^{p} R(h_j)}$$

2.6 Comparison: Proportionate vs. Rank Selection

To illustrate the difference, consider a population of 4 individuals where one is significantly better than the others:

Individual	**Fitness**	**Prop. Prob.**	**Rank (Prob.)**
h_1 (Best)	100	77%	Rank 3 (37.5%)
h_2	10	8%	Rank 2 (25.0%)
h_3	10	8%	Rank 2 (25.0%)
h_4 (Worst)	5	4%	Rank 1 (12.5%)

Analysis: In Fitness Proportionate Selection, h_1 is nearly **10 times** more likely to be selected than h_2. In Rank Selection, despite the massive fitness gap, h_1 is only **1.5 times** more likely to be selected than h_2.

2.7 Advantages

1. **Controlled Selection Pressure:** By using ranks, the algorithm maintains a steady pace of evolution, preventing a few individuals from taking over the gene pool too quickly.
2. **Robustness:** It is not sensitive to the specific scale of the fitness function (e.g., whether fitness is measured in units of 10 or 1,000).

3. **Diversity Preservation:** It allows "weaker" individuals to stay in the population longer, preserving their unique bit patterns for potential future recombination.

2.8 Crossover (Recombination)

Crossover combines "genetic material" from two parents.

1. **Single-Point Crossover:** A random index is chosen.
   - Parent A: 111|11
   - Parent B: 000|00
   - Offspring: 11100 and 00011
2. **Two-Point Crossover:** Swaps the middle segment.
   - Parent A: 11|00|11
   - Parent B: 00|11|00
   - Offspring: 111111 and 000000
3. **Uniform Crossover (Masking):** A bit-mask is generated. If mask bit is 1, take from Parent A; if 0, take from Parent B.
   - Parent A: `11111`
   - Parent B: `00000`
   - Mask: `10101`
   - Offspring: `10101`

2.9 Mutation

Mutation prevents the population from converging on a local optimum by flipping random bits.

- **Example:** String 11011 with mutation at pos 3 becomes 11111.

3 The Prototypical Genetic Algorithm

The GA treats learning as an optimization problem, aiming to maximize a *Fitness* function.

3.1 Algorithm Parameters

- p **(Population Size):** The number of hypotheses in the population.
- r **(Replacement Rate):** The fraction of the population replaced by crossover.
- m **(Mutation Rate):** The probability that a bit will be mutated.

3.2 Formal Algorithm Definition

GA$(Fitness, Fitness\_threshold, p, r, m)$
Initialize: $P \leftarrow p$ random hypotheses
Evaluate: For each $h \in P$, compute $Fitness(h)$
while $[\max_h Fitness(h)] < Fitness\_threshold$ **do**
1. **Select:** Probabilistically select $(1-r)p$ members to P_S via Fitness Proportionate Selection.
2. **Crossover:** Select $\frac{r \cdot p}{2}$ pairs. Apply crossover to produce two offspring and add to P_S.
3. **Mutate:** Invert a random bit in $m \cdot p$ random members of P_S.
4. **Update:** $P \leftarrow P_S$
5. **Evaluate:** For each h in P, compute $Fitness(h)$

end while
return highest fitness hypothesis in P

3.3 The Prototypical GA: Step-by-Step Example

Problem: Find the bit string that maximizes the "One-Max" function (count of 1s). **Params:** $p = 4$, $r = 0.5$ (50% crossover), $m = 0.01$.

1. **Initialize:** $P = \{h_1 : 10001, h_2 : 01010, h_3 : 11000, h_4 : 00010\}$
2. **Evaluate Fitness:** $Fitness(h_1) = 2, Fitness(h_2) = 2, Fitness(h_3) = 2, Fitness(h_4) = 1$. Total = 7.
3. **Select:** Probabilities: $h_1 = 2/7, h_2 = 2/7, h_3 = 2/7, h_4 = 1/7$. Let's say h_1, h_2, h_3 are selected for P_s.
4. **Crossover:** Select pairs for replacement (50% of 4 = 2 offspring). Swap h_1 and h_3 at pos 2: $h_1 : 10|001$, $h_3 : 11|000 \rightarrow$ Offspring: $10000, 11001$.
5. **Mutate:** Flip a bit in a random string: $11001 \rightarrow 11101$.
6. **Repeat:** New population is evaluated until a string of all 1s appears.

3.4 GABIL: Learning Rule Sets

GABIL is a system that uses Genetic Algorithms to learn sets of propositional rules. It treats the problem of learning rules as an optimization task, evolving a population of hypotheses that compete based on their classification accuracy.

3.5 Knowledge Representation

In GABIL, each hypothesis is represented as a bit string that encodes a set of rules (a disjunction of rules).

3.5.1 Encoding a Single Attribute

Each attribute is represented by a bit string where each bit corresponds to a possible value of that attribute. A bit is set to 1 if that value is permitted by the rule.

Example: Consider the attribute *Outlook* with values $\{Sunny, Overcast, Rain\}$.

- `100` represents the constraint $Outlook = Sunny$.
- `011` represents the constraint $Outlook = Overcast \vee Rain$.
- `111` represents a "don't care" condition (any value is allowed).

3.5.2 Encoding a Rule

A single rule is formed by concatenating the bit strings for all attributes, followed by the target classification bit.

Example Rule: `IF (Outlook = Rain) AND (Wind = Strong) THEN (PlayTennis = No)` Assuming *Outlook* has 3 values, *Wind* has 2 values $\{Strong, Weak\}$, and *PlayTennis* is binary:

Outlook	Wind	PlayTennis
001	10	0

The resulting bit string for this rule is `001100`.

3.5.3 Encoding a Rule Set

GABIL represents a *set* of rules by concatenating multiple rules into a single, longer bit string.

$$Hypothesish = Rule_1 Rule_2 \dots Rule_n$$

3.6 Genetic Operators in GABIL

GABIL employs standard selection and mutation, but it uses a specialized version of crossover to handle the multi-rule structure.

4 Variable-Length Crossover in GABIL

In GABIL, a hypothesis h consists of a set of rules, each of length K. Because different hypotheses can have a different number of rules, crossover must be able to combine strings of different lengths while maintaining the semantic integrity of the rule attributes. If an attribute has 3 possible values, it is represented by 3 bits.If the rule doesn't care about that attribute, all 3 bits are set to 1 (e.g., 111).This ensures that every single rule in the population has the exact same structure and length K, even if some parts of the string are effectively "empty" or "ignored."

4.1 The Alignment Constraint

To ensure that offspring are semantically valid, the crossover points d_1 (for parent h_1) and d_2 (for parent h_2) must be aligned:

1. Pick d_1 at any bit boundary in h_1.
2. Pick d_2 such that $d_2 \equiv d_1 \pmod{K}$.

4.2 Numerical Example

Let $K = 6$ (3 bits for Outlook, 2 for Wind, 1 for Target).

- **Parent 1 (h_1):** `110101 011010` (2 rules, length 12)
- **Parent 2 (h_2):** `100111` (1 rule, length 6)

If we choose $d_1 = 8$, then $d_1 \pmod 6 = 2$. Therefore, d_2 must be 2.

- h_1 is split as: `11010101 | 1010`
- h_2 is split as: `10 | 0111`

The resulting offspring are:

- **Offspring 1:** `11010101` + `0111` $\rightarrow$ `110101 010111` (2 rules)
- **Offspring 2:** `10` + `1010` $\rightarrow$ `101010` (1 rule)

Note that the attribute boundaries remain intact in both offspring.

4.3 Extension Operators

GABIL includes two specialized operators to perform generalization:

- **AddAlternative:** A randomly selected 0 bit is changed to 1. This generalizes the rule by adding a permitted value to an attribute constraint.
- **DropCondition:** All bits for a chosen attribute in a rule are set to 1. This effectively removes the attribute from the precondition, making the rule more general.

4.4 Evolving the Learning Strategy

A unique feature of GABIL is that it can evolve its own learning parameters. Each hypothesis bit string can be extended with two additional bits: **AA** (AddAlternative) and **DC** (DropCondition).

- If the **AA** bit is 1, the *AddAlternative* operator is permitted for this individual.
- If 0, the operator is disabled.

This allows the GA to discover which generalization operators are most effective for the specific dataset being learned.

4.5 Fitness Function

The fitness of a rule set h is defined based on its classification performance:

$$Fitness(h) = (correct(h))^2$$

Squaring the accuracy provides a higher selective advantage to nearly perfect hypotheses, accelerating convergence in the later stages of evolution.

4.6 Performance

Experimental results show that GABIL is competitive with symbolic learning methods. In a study of 12 synthetic problems:

- **GABIL (Standard):** 92.1% accuracy.
- **GABIL (With AA/DC):** 95.2% accuracy.
- **C4.5/AQ14:** Ranged from 91.2% to 96.6% accuracy.

Variable Length: Rule sets can have different numbers of rules. GABIL uses "Semantically valid" crossover, ensuring cuts only happen between full rules or specific attribute boundaries.

4.7 Genetic Programming (GP)

Genetic Programming (GP) is a significant departure from standard Genetic Algorithms. Instead of searching for optimal parameters within a fixed string, GP searches for an optimal **executable program**.

4.8 Representation: Programs as Trees

In GP, programs are represented as hierarchical tree structures. This allows the GA to explore hypotheses of varying complexity and length.

- **Functions (Internal Nodes):** Arithmetic operators ($+, \times$), logical operators, or domain-specific functions.
- **Terminals (Leaves):** Constants or variables relevant to the problem.

4.9 GP Crossover: Subtree Swapping

Crossover in GP is performed by selecting a random node in each of two parent trees and swapping the entire subtrees rooted at those nodes. This allows for the recombination of functional logic "building blocks."

4.9.1 TikZ Diagram: GP Crossover

The following TikZ code illustrates two parents swapping subtrees to produce a new offspring.

4.10 GP Mutation

Mutation in GP involves selecting a random node and replacing the subtree rooted there with a completely new, randomly generated subtree. This ensures the search does not prematurely converge on a local optimum by introducing entirely new logic.

4.11 Summary of Genetic Programming

Unlike traditional GAs that optimize static parameters, GP optimizes **processes**. It is uniquely suited for tasks where the structure of the solution (the program's logic) is unknown at the start.

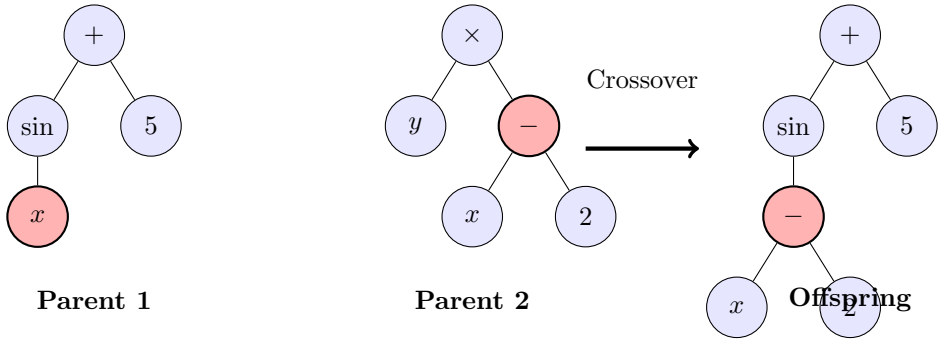

Figure 1: GP Crossover: The subtree $(x - 2)$ from Parent 2 replaces the node x in Parent 1.

4.12 Summary

GAs are parallel, robust, and effective for complex optimization. They are particularly useful when the fitness landscape is "jagged" or when the learning task requires discovering structural logic.